

Atelier Machine Learning

Analyse Comportementale Clientèle Retail

COMPTE RENDU

Atelier Pratique : E-commerce de Cadeaux

Préparé par Rouag Dhia

Module	Machine Learning - GI2
Encadrant	Mme Fadoua Drira
Dataset	retail_customers_COMPLETE_CATEGORICAL.csv
Algorithmes	Logistic Regression, Random Forest, K-Means, DBSCAN
Langage / Outils	Python 3, Scikit-learn, Flask, Pandas
Année universitaire	2025-2026

Année Universitaire : 2025-2026

Table des matières

1	Introduction Générale	2
2	Introduction générale	2
2.1	Contexte	2
2.2	Problématique	2
2.3	Objectifs du projet	2
2.4	Structure du projet	3
2.5	Pipeline et méthodologie	3
3	Cadre Général et Concepts de Base	4
3.1	Le Machine Learning supervisé et non supervisé	4
3.2	La classification	5
3.3	Les métriques de classification	5
3.4	Le clustering	5
3.5	La régression	6
3.6	Les métriques de régression	6
4	Exploration et Préparation des Données	7
4.1	Exploration des données	7
4.2	Prétraitement des données	10
5	Modélisation – Classification du Churn	12
5.1	Objectif de la classification	12
5.2	Modèles utilisés	12
5.3	Résultats des modèles	12
5.4	Matrices de confusion	12
5.5	Courbes ROC	14
5.6	Importance des variables	16
5.7	Modèle final retenu	17
5.8	Importance des variables	17
6	Modélisation – Clustering (Segmentation)	18
6.1	K-Means – Sélection du nombre optimal de clusters	18
6.2	Profils des clusters K-Means	18
6.3	DBSCAN – Détection de clusters et d'outliers	19
7	Modélisation – Régression (Dépense Future)	21
7.1	Objectif et préparation	21
7.2	Comparaison des modèles de régression	21
7.3	Valeurs réelles vs valeurs prédites	22
7.4	Importance des variables pour la régression	22
7.5	Exemples de prédictions	23
8	Déploiement – Interface Flask	24
8.1	Architecture de l'application	24
8.2	Dashboard web	24
8.3	Gestion des colonnes	24
8.4	Lancement de l'application	25
9	Conclusion	26
10	Références	27

1 Introduction Générale

2 Introduction générale

2.1 Contexte

Dans un marché e-commerce de plus en plus concurrentiel, la compréhension du comportement client constitue un enjeu stratégique majeur pour les entreprises. La fidélisation des clients existants permet non seulement de préserver le chiffre d'affaires, mais également de réduire les coûts liés à l'acquisition de nouveaux clients.

Dans ce contexte, une entreprise spécialisée dans la vente de cadeaux en ligne cherche à mieux exploiter ses données clients afin d'anticiper les comportements à risque, d'identifier des profils homogènes et d'estimer la valeur commerciale des clients. Le Machine Learning apparaît alors comme une solution pertinente pour transformer les données disponibles en informations utiles pour la prise de décision marketing et commerciale.

Ce projet s'inscrit dans le cadre de l'atelier pratique de Machine Learning du module GI2, année universitaire 2025–2026. Il vise à mettre en place une chaîne complète de traitement des données, depuis l'exploration initiale jusqu'à la modélisation et au déploiement d'une interface web interactive exploitant les modèles développés.

2.2 Problématique

L'entreprise dispose d'un dataset composé de 4372 clients décrits par 52 variables transactionnelles, comportementales et complémentaires. Ces données sont volontairement imparfaites afin de simuler un cas réaliste de projet de Machine Learning. Elles présentent plusieurs problèmes de qualité qui doivent être traités avant toute phase de modélisation.

Parmi les principales difficultés identifiées, on peut citer :

- la présence de valeurs manquantes dans certaines variables, notamment **Age** ;
- l'existence de valeurs aberrantes dans des variables comme **SupportTicketsCount** et **SatisfactionScore** ;
- des formats hétérogènes pour certaines informations, notamment **RegistrationDate** ;
- des variables inutiles ou constantes, comme **CustomerID** et **NewsletterSubscribed** ;
- des variables à risque de fuite d'information, telles que **ChurnRiskCategory**, **RFMSegment**, **CustomerType**, **LoyaltyLevel**, **SpendingCategory** et **AccountStatus** ;
- un déséquilibre modéré de la variable cible **Churn**.

À partir de ce contexte, le projet cherche à répondre à trois questions principales :

- quels clients risquent de quitter la plateforme ? (**classification**) ;
- quels groupes homogènes de clients peut-on identifier ? (**clustering**) ;
- quelle dépense totale peut-on estimer pour un client ? (**régression**).

Ces problématiques justifient l'utilisation de plusieurs approches de Machine Learning afin d'extraire des connaissances utiles à partir des données disponibles et d'aider l'entreprise à mieux comprendre, segmenter et prédire le comportement de sa clientèle.

2.3 Objectifs du projet

Les objectifs de ce projet sont à la fois pédagogiques et métier. Ils couvrent l'ensemble du cycle de vie d'un projet de Machine Learning, depuis l'analyse des données jusqu'à l'exploitation des modèles.

Les objectifs principaux sont les suivants :

- explorer les données brutes afin d'identifier leur structure, leurs distributions et leurs problèmes de qualité ;
- nettoyer les données en traitant les valeurs manquantes, les valeurs aberrantes, les formats inconsistants et les colonnes inutiles ;

- réaliser un *feature engineering* afin de transformer certaines données brutes, comme les dates et les adresses IP, en variables exploitables ;
- encoder les variables catégorielles et normaliser les variables numériques avant la modélisation ;
- utiliser l'Analyse en Composantes Principales (ACP) comme outil d'exploration et de visualisation des données multidimensionnelles ;
- appliquer des méthodes de clustering afin de segmenter les clients selon leurs comportements ;
- entraîner et comparer plusieurs modèles de classification pour prédire le churn des clients ;
- entraîner et comparer des modèles de régression pour estimer la dépense totale des clients ;
- évaluer les modèles à l'aide de métriques adaptées à chaque tâche ;
- développer une interface web avec Flask permettant d'exploiter les modèles sauvegardés.

2.4 Structure du projet

Le projet suit une architecture modulaire en Python. Cette organisation permet de séparer clairement les différentes étapes : exploration, preprocessing, modélisation, évaluation, prédiction et déploiement.

```

1 projet_ml_retail/
2   app/
3     app.py                --> API REST Flask + Dashboard
4     templates/index.html  --> Interface web interactive
5
6   data/
7     raw/                  --> Dataset brut
8     processed/            --> Dataset nettoyé
9     train_test/           --> Jeux X_train, X_test, y_train, y_test
10
11  models/                  --> Modeles sauvegardés (.joblib)
12
13  notebooks/
14    exploration.ipynb      --> Analyse exploratoire des données
15
16  reports/
17    figures/               --> Figures générées automatiquement
18
19  src/
20    preprocessing.py       --> Nettoyage, feature engineering, split
21    train_model.py         --> Classification du churn
22    clustering.py           --> Segmentation des clients
23    regression.py          --> Prédiction de MonetaryTotal
24    predict.py             --> Prédiction batch
25    utils.py               --> Fonctions utilitaires communes
26
27  requirements.txt
28  README.md
29  .gitignore

```

Listing 1 – Arborescence générale du projet

Cette structure facilite la lisibilité du projet, la maintenance du code et la réutilisation des différents composants. Les modèles entraînés sont sauvegardés dans le dossier `models/` au format `joblib`, ce qui permet leur réutilisation dans l'application Flask.

2.5 Pipeline et méthodologie

La méthodologie adoptée suit une chaîne complète de traitement en Machine Learning. Elle commence par l'analyse exploratoire des données, se poursuit par le nettoyage et la préparation des variables, puis aboutit à l'entraînement, l'évaluation et le déploiement des modèles.

Pipeline complet du projet

Exploration des données (EDA) → Préparation et preprocessing → Modélisation : clustering, classification et régression → Évaluation des performances → Déploiement avec Flask API et Dashboard

Les principales étapes du projet sont les suivantes :

- **Étape 1 : Exploration des données** avec le notebook `exploration.ipynb`, afin d'analyser les distributions, les corrélations, les valeurs manquantes et les anomalies ;
- **Étape 2 : Preprocessing** avec `src/preprocessing.py`, comprenant le nettoyage, l'imputation, la normalisation, l'encodage des variables catégorielles et la suppression des variables à risque de data leakage ;
- **Étape 3 : Clustering** avec `src/clustering.py`, afin de segmenter les clients à l'aide de K-Means et de compléter l'analyse avec DBSCAN pour la détection de micro-segments et d'outliers ;
- **Étape 4 : Classification** avec `src/train_model.py`, afin de prédire la variable `Churn` à l'aide de modèles supervisés tels que Logistic Regression, Random Forest et XGBoost ;
- **Étape 5 : Régression** avec `src/regression.py`, afin d'estimer la variable `MonetaryTotal` à l'aide de Linear Regression et Random Forest Regressor ;
- **Étape 6 : Prédiction batch** avec `src/predict.py`, permettant d'appliquer le modèle de classification sauvegardé sur un fichier CSV ;
- **Étape 7 : Déploiement** avec `app/app.py`, sous forme d'API REST Flask et de tableau de bord interactif.

La méthodologie respecte les bonnes pratiques du Machine Learning :

- séparation des données en ensembles d'apprentissage et de test selon une répartition 80% / 20% ;
- utilisation d'un split stratifié pour conserver la même proportion de classes dans les jeux d'entraînement et de test ;
- ajustement du préprocesseur uniquement sur le jeu d'entraînement afin d'éviter le *data leakage* ;
- gestion du déséquilibre de classes par pondération avec `class_weight="balanced"` et `scale_pos_weight` pour XGBoost ;
- suppression des variables trop proches de la cible afin d'éviter des performances artificiellement élevées ;
- sélection des modèles à l'aide de métriques adaptées : F1-score et ROC-AUC pour la classification, MAE/RMSE/ R^2 pour la régression et score Silhouette pour le clustering ;
- sauvegarde des modèles et preprocessors avec `joblib` afin de garantir leur réutilisation dans l'interface Flask.

Cette démarche permet d'obtenir une solution complète, allant de l'analyse des données à l'exploitation pratique des modèles dans une interface utilisateur.

3 Cadre Général et Concepts de Base

3.1 Le Machine Learning supervisé et non supervisé

Le Machine Learning, ou apprentissage automatique, regroupe un ensemble de méthodes permettant à un système informatique d'apprendre à partir de données, sans être explicitement programmé pour chaque tâche. Dans le cadre de ce projet, deux grandes approches sont mobilisées :

- **le Machine Learning supervisé** : l'algorithme apprend à partir d'exemples étiquetés, c'est-à-dire pour lesquels la sortie attendue est connue. Cette approche est utilisée dans ce projet pour la **classification** et la **régression** ;
- **le Machine Learning non supervisé** : l'algorithme cherche à découvrir des structures ou des regroupements cachés dans les données, sans disposer de variable cible. Cette approche est utilisée pour le **clustering**.

3.2 La classification

La classification est une tâche d'apprentissage supervisé qui consiste à prédire l'appartenance d'un individu à une catégorie discrète. Dans ce projet, elle vise à répondre à la question suivante : « ce client va-t-il quitter l'entreprise ($Churn = 1$) ou rester fidèle ($Churn = 0$) ? »

La régression logistique

La régression logistique est un modèle de classification binaire qui estime la probabilité d'appartenance à la classe positive à l'aide d'une fonction sigmoïde. Elle constitue une méthode simple, rapide et interprétable, souvent utilisée comme modèle de référence.

$$P(Churn = 1 | x) = \frac{1}{1 + e^{-(\beta_0 + \beta_1 x_1 + \dots + \beta_p x_p)}}$$

Malgré sa simplicité, ce modèle reste limité lorsque la séparation entre les classes n'est pas linéaire.

Random Forest Classifier

Le modèle Random Forest repose sur un ensemble d'arbres de décision entraînés sur différents sous-échantillons aléatoires des données, selon le principe du *bagging*. La prédiction finale est obtenue par vote majoritaire des arbres.

Ce modèle présente plusieurs avantages : il est robuste aux valeurs aberrantes, capable de modéliser des relations non linéaires et permet d'estimer l'importance relative des variables.

3.3 Les métriques de classification

Plusieurs métriques sont utilisées pour évaluer les performances des modèles de classification.

TABLE 1 – Métriques d'évaluation de la classification

Métrique	Définition	Formule
Accuracy	Proportion de prédictions correctes	$\frac{TP+TN}{TP+TN+FP+FN}$
Precision	Proportion de vrais churners parmi les alertes	$\frac{TP}{TP+FP}$
Recall	Proportion de churners correctement détectés	$\frac{TP}{TP+FN}$
F1-score	Moyenne harmonique entre précision et rappel	$\frac{2 \times Precision \times Recall}{Precision + Recall}$
ROC-AUC	Capacité de discrimination du modèle	Aire sous la courbe ROC

Le F1-score constitue un critère particulièrement pertinent dans ce projet, car il permet d'équilibrer la précision et le rappel dans un contexte de classes déséquilibrées.

3.4 Le clustering

Le clustering est une tâche d'apprentissage non supervisé qui consiste à regrouper les données en ensembles homogènes, sans étiquettes prédéfinies. Dans ce projet, il permet de répondre à la question suivante : « quels groupes homogènes de clients peut-on identifier ? »

K-Means

K-Means est un algorithme de partitionnement qui répartit les données en k clusters en minimisant l'inertie intra-cluster, c'est-à-dire la somme des distances entre les points et leur centroïde.

Le choix du nombre optimal de clusters peut être guidé par plusieurs critères :

- la méthode du coude (*Elbow*) ;
- le score de silhouette ;
- l'indice de Davies-Bouldin.

DBSCAN

DBSCAN (*Density-Based Spatial Clustering of Applications with Noise*) est un algorithme de clustering fondé sur la densité. Il permet de détecter des clusters de forme libre et d'identifier les observations atypiques, considérées comme du bruit.

Contrairement à K-Means, DBSCAN ne nécessite pas de fixer à l'avance le nombre de clusters, ce qui le rend particulièrement utile pour détecter des structures plus complexes dans les données.

3.5 La régression

La régression est une tâche d'apprentissage supervisé visant à prédire une variable numérique continue. Dans ce projet, elle est utilisée pour répondre à la question suivante : « quelle sera la dépense future (*MonetaryTotal*) d'un client ? »

Régression linéaire

La régression linéaire modélise la relation entre une variable cible continue et un ensemble de variables explicatives à l'aide d'une fonction linéaire. Elle constitue une approche simple servant souvent de modèle de base de comparaison.

Random Forest Regressor

Le modèle Random Forest Regressor repose sur le même principe que la version utilisée en classification, mais il produit cette fois une valeur numérique continue. La prédiction finale correspond à la moyenne des prédictions générées par les différents arbres.

3.6 Les métriques de régression

Les performances des modèles de régression sont évaluées à l'aide des métriques suivantes :

TABLE 2 – Métriques d'évaluation de la régression

Métrique	Définition	Interprétation
MAE	Erreur absolue moyenne	Mesure l'écart moyen entre valeurs réelles et prédites
RMSE	Racine de l'erreur quadratique moyenne	Pénalise davantage les erreurs importantes
R^2	Coefficient de détermination	Mesure la part de variance expliquée par le modèle

4 Exploration et Préparation des Données

4.1 Exploration des données

Présentation du dataset

Le dataset `retail_customers_COMPLETE_CATEGORICAL.csv` contient 4372 clients et 52 variables issues de transactions réelles, volontairement imparfaites afin de simuler des conditions de travail réalistes.

TABLE 3 – Répartition des variables du dataset

Type de variables	Nombre	Exemples
Numériques continues	17	MonetaryTotal, Age, ReturnRatio, WeekendRatio
Numériques entières	17	Recency, Frequency, TotalTrans, UniqueProducts
Catégorielles	17	RFMSegment, CustomerType, Region, LoyaltyLevel
Variable cible	1	Churn (0 = fidèle, 1 = parti)
Total	52	Ensemble des variables du dataset

Distribution de la variable cible Churn

L'exploration réalisée dans `notebooks/exploration.ipynb` met en évidence la distribution de la variable cible **Churn**.

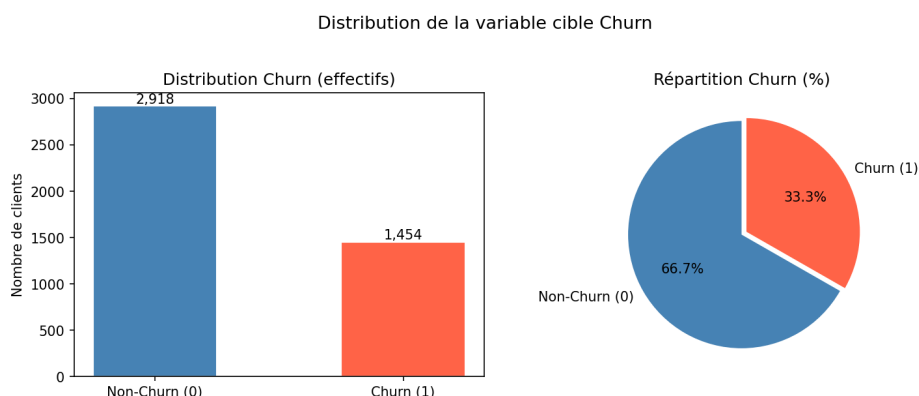


FIGURE 1 – Distribution de la variable cible Churn

Le dataset présente un déséquilibre de classes avec environ 66.7% de clients fidèles contre 33.3% de clients partis. Ce déséquilibre justifie l'utilisation d'une stratégie d'équilibrage dans les modèles de classification.

Analyse du churn par segment

L'analyse du churn par segment met en évidence des différences significatives entre plusieurs profils de clients.

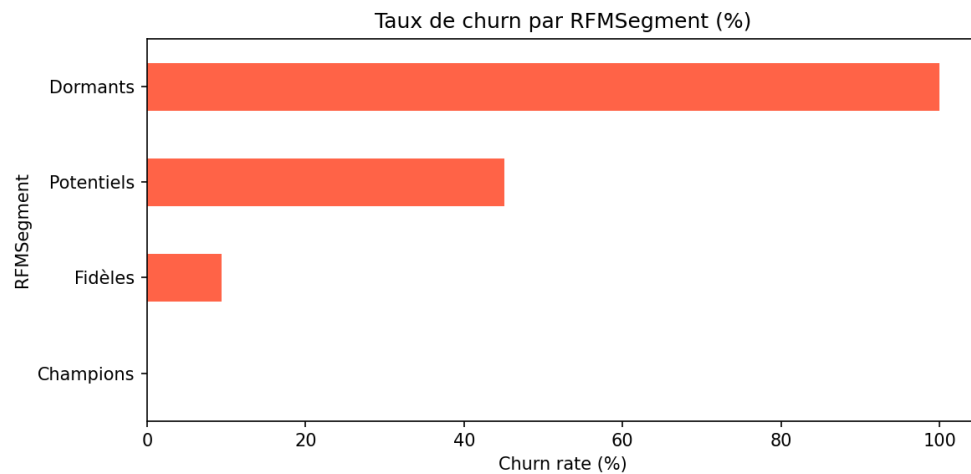


FIGURE 2 – Taux de churn par segment RFM

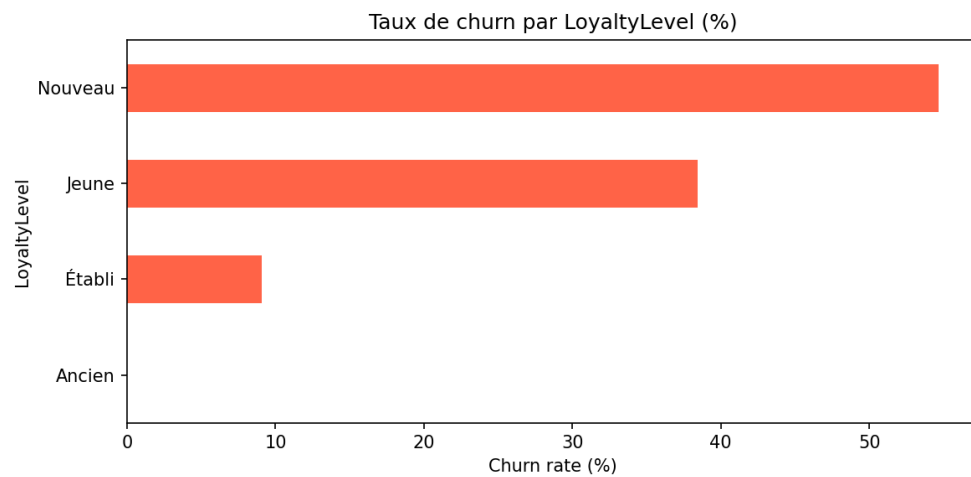


FIGURE 3 – Taux de churn par niveau de fidélité

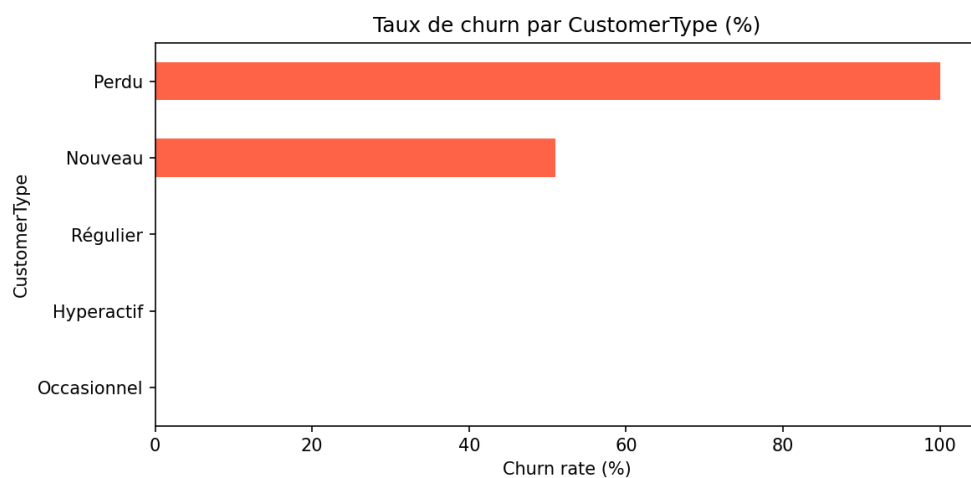


FIGURE 4 – Taux de churn par type de client

Ces résultats montrent que le risque de churn varie fortement selon le profil du client. Les segments les plus fragiles, tels que les clients perdus ou nouvellement acquis, présentent des taux de churn nettement plus élevés. À l'inverse, les profils plus réguliers ou plus engagés apparaissent globalement plus

stables. Cette observation confirme que les variables de segmentation client apportent une information utile pour anticiper le départ des clients.

Matrice de corrélation

La matrice de corrélation permet d'identifier les relations entre les variables numériques et de mettre en évidence d'éventuelles redondances ou situations de multicollinéarité.

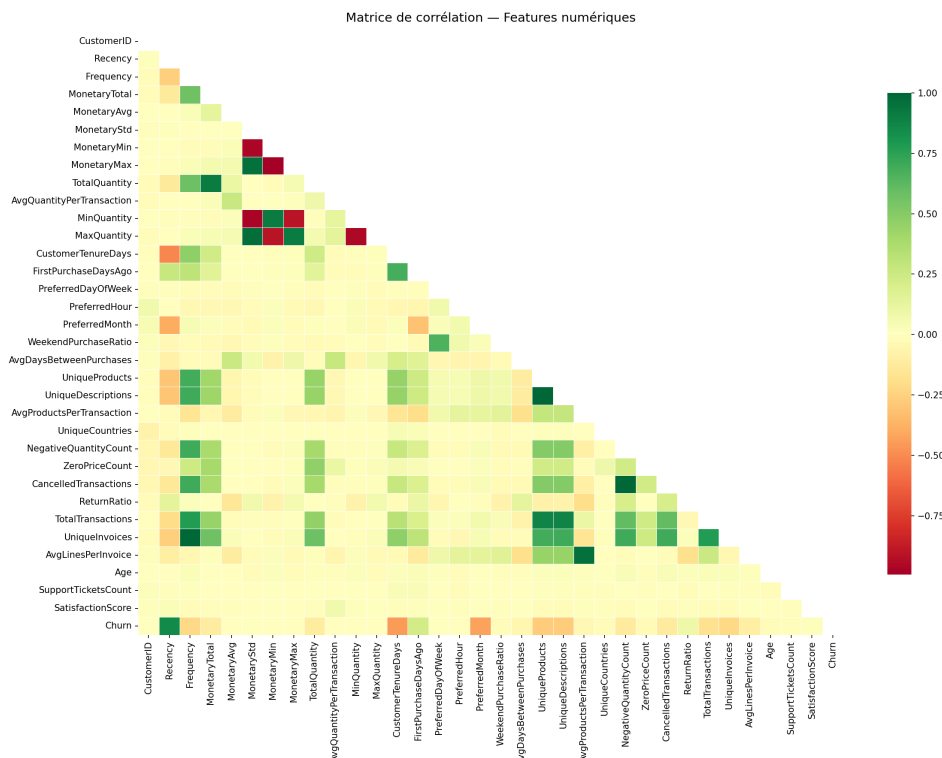


FIGURE 5 – Matrice de corrélation des variables numériques

L'analyse met en évidence des corrélations importantes entre plusieurs variables monétaires et transactionnelles, ce qui confirme la nécessité d'examiner attentivement la redondance de certaines informations avant la modélisation.

Analyse en composantes principales (ACP)

Une Analyse en Composantes Principales (ACP) a été appliquée sur les variables numériques après imputation et normalisation afin de réduire la dimension des données et de faciliter leur visualisation.

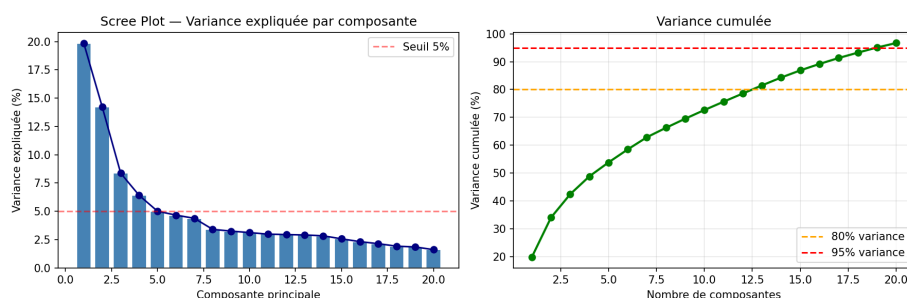


FIGURE 6 – Scree plot de l'ACP

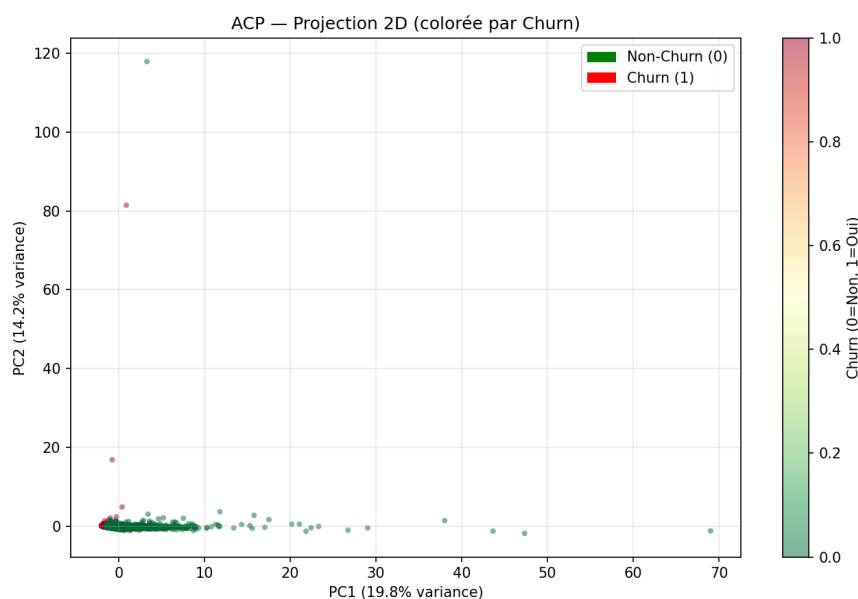


FIGURE 7 – Projection ACP 2D colorée par churn

L'ACP montre que les classes ne sont pas parfaitement séparables de manière linéaire, ce qui justifie l'utilisation de modèles plus flexibles pour la classification.

4.2 Prétraitement des données

Problèmes de qualité identifiés

Avant la modélisation, une analyse de la qualité des données a été menée afin d'identifier les principales anomalies pouvant nuire aux performances des modèles.

TABLE 4 – Catalogue des problèmes de qualité et solutions appliquées

Problème	Feature(s)	Fréquence	Solution appliquée	Fichier
Valeurs manquantes	Age	~30%	SimpleImputer (médiane)	utils.py
Valeurs aberrantes	Satisfaction (-1, 0, 99)	5–8%	<code>clean_domain_anomalies()</code>	utils.py
Valeurs aberrantes	SupportTickets (-1, 999)	5–8%	<code>clean_domain_anomalies()</code>	utils.py
Format date brut	RegistrationDate	100%	<code>parse_registration_date()</code>	utils.py
Feature constante	Newsletter (= Yes)	100%	<code>drop_constant_columns()</code>	utils.py
Data leakage	ChurnRiskCategory	—	Suppression	preprocessing.py
Identifiant inutile	CustomerID	—	Suppression	preprocessing.py
Déséquilibre des classes	Churn (~2 :1)	—	<code>class_weight='balanced'</code>	train_model.py

Feature engineering

Afin d'enrichir les données initiales, plusieurs variables dérivées ont été créées pour améliorer la capacité descriptive et prédictive du dataset.

TABLE 5 – Variables dérivées créées par feature engineering

Nouvelle feature	Formule / origine	Intérêt métier
MonetaryPerDay	MonetaryTotal / (Recency + 1)	Intensité monétaire récente
AvgBasketValue	MonetaryTotal / Frequency	Valeur typique d'une commande
TenureRatio	CustomerTenure / (Recency + 1)	Ancienneté vs inactivité récente
FrequencyIntensity	Frequency / CustomerTenure	Commandes par jour d'ancienneté
IP_IsPrivate	Détection des plages RFC 1918	Connexion réseau privé / public
IP_FirstOctet	Premier octet de l'IP	Indicateur réseau
RegistrationYear/Month	Depuis RegistrationDate	Saisonnalité d'inscription
CustomerTenureDays	Ancienneté en jours	Durée précise de la relation client

Pipeline de preprocessing

Le pipeline de prétraitement a été conçu afin de garantir une préparation rigoureuse des données tout en évitant les fuites d'information entre apprentissage et test.

Pipeline de preprocessing

1. création de nouvelles variables via `add_ip_features()`, `add_rfm_features()` et `parse_registration_date()` ;
2. correction des anomalies métier avec `clean_domain_anomalies()` ;
3. suppression des colonnes inutiles ou à risque de fuite d'information ;
4. séparation stratifiée des données en 80% apprentissage et 20% test ;
5. construction du préprocesseur :
 - variables numériques : `SimpleImputer(strategy='median') + StandardScaler` ;
 - variables catégorielles : `SimpleImputer(strategy='most_frequent') + OneHotEncoder(handle_unknown='ignore')` ;
6. sauvegarde des artefacts : `preprocessor.joblib`, `X_train.csv`, `X_test.csv`, `y_train.csv`, `y_test.csv`.

5 Modélisation – Classification du Churn

5.1 Objectif de la classification

La classification a pour objectif de prédire si un client risque de quitter la plateforme ou non. La variable cible utilisée est **Churn**, avec deux classes :

- **Churn** = 0 : client fidèle ;
- **Churn** = 1 : client parti.

Après les premières expérimentations, certains résultats étaient presque parfaits, ce qui indiquait un risque de *data leakage*. Certaines variables trop proches de la cible, telles que **ChurnRiskCategory**, **RFMSegment**, **CustomerType**, **LoyaltyLevel**, **SpendingCategory** et **AccountStatus**, ont donc été supprimées afin d'éviter que le modèle n'apprenne des raccourcis artificiels au lieu des vrais comportements clients.

5.2 Modèles utilisés

Le script `src/train_model.py` entraîne trois modèles de classification sur le jeu d'apprentissage, puis les évalue sur le jeu de test. Le critère principal de sélection retenu est le **F1-score**, car il permet d'équilibrer la précision et le rappel dans un contexte de classes modérément déséquilibrées.

Les modèles utilisés sont :

- **Logistic Regression** : modèle linéaire simple et interprétable, utilisé avec `max_iter=1000`, `class_weight="balanced"` et `random_state=42` ;
- **Random Forest Classifier** : ensemble d'arbres de décision, capable de modéliser des relations non linéaires, utilisé avec `n_estimators=300`, `class_weight="balanced"` et `random_state=42` ;
- **XGBoost Classifier** : modèle de boosting basé sur des arbres de décision construits séquentiellement. Il est utilisé avec `n_estimators=300`, `max_depth=6`, `learning_rate=0.1`, `subsample=0.8`, `colsample_bytree=0.8` et `scale_pos_weight` pour gérer le déséquilibre des classes.

5.3 Résultats des modèles

Les performances obtenues sur le jeu de test sont résumées dans le tableau suivant :

TABLE 6 – Comparaison des métriques de classification sur le jeu de test

Modèle	Accuracy	Précision	Recall	F1-score	ROC-AUC	Sélection
Logistic Regression	0.8903	0.7893	0.9141	0.8471	0.9592	—
Random Forest	0.9166	0.9258	0.8144	0.8665	0.9743	—
XGBoost	0.9703	0.9715	0.9381	0.9545	0.9952	Retenu

Les résultats montrent que le modèle **XGBoost** obtient les meilleures performances globales. Il présente le meilleur **F1-score** avec une valeur de **0.9545**, ainsi qu'un **ROC-AUC** de **0.9952**. Ce modèle offre donc le meilleur équilibre entre la précision et la détection des clients churners, ce qui justifie son choix comme modèle final de classification.

5.4 Matrices de confusion

Les matrices de confusion permettent d'observer la répartition des prédictions correctes et incorrectes à travers les vrais positifs, les vrais négatifs, les faux positifs et les faux négatifs.

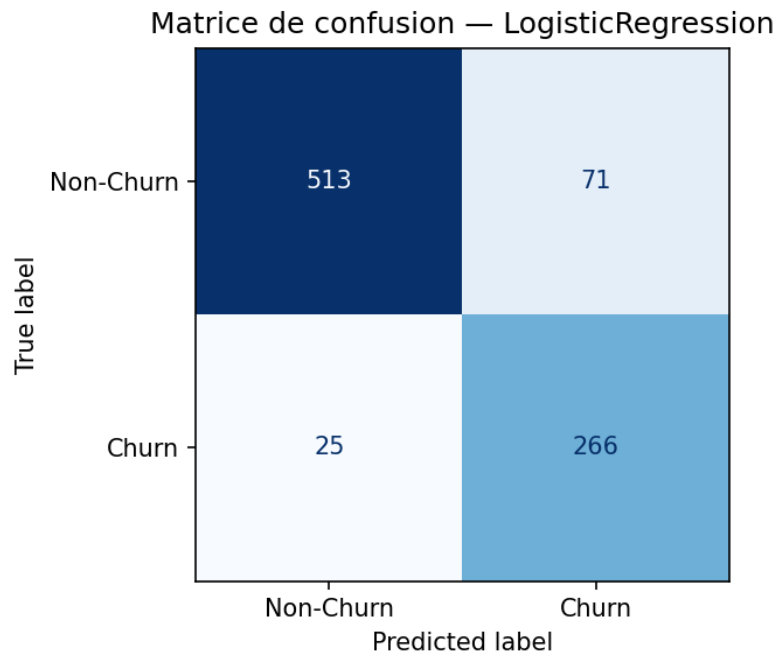


FIGURE 8 – Matrice de confusion – Logistic Regression

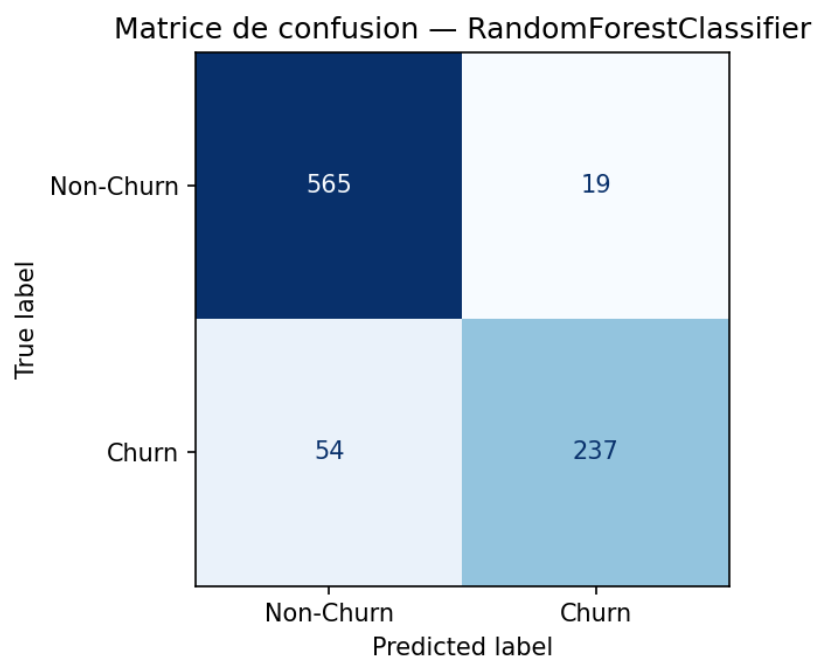


FIGURE 9 – Matrice de confusion – Random Forest Classifier

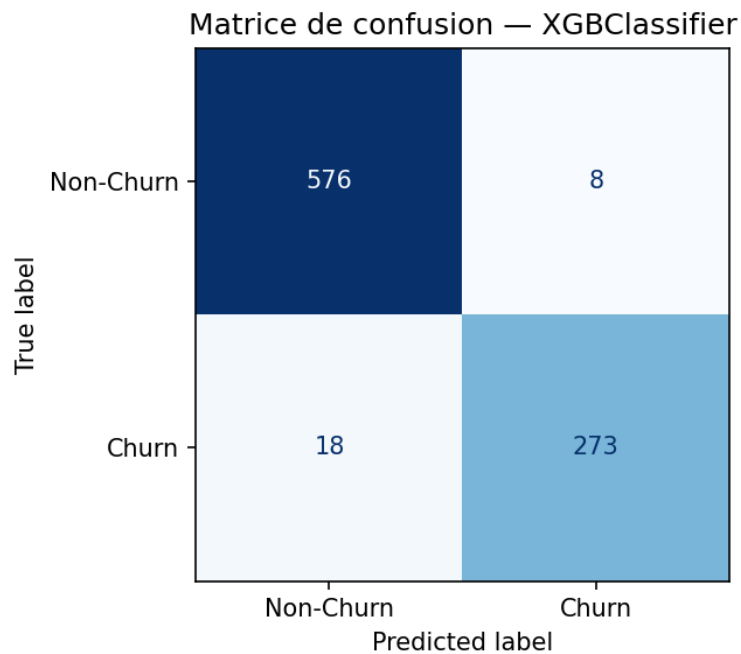


FIGURE 10 – Matrice de confusion – XGBoost Classifier

La matrice de confusion de XGBoost montre une meilleure capacité à détecter les clients churners tout en limitant les fausses alertes. Comparé aux deux autres modèles, XGBoost présente un meilleur équilibre entre les faux positifs et les faux négatifs, ce qui confirme les résultats obtenus avec le F1-score.

5.5 Courbes ROC

La courbe ROC illustre le compromis entre le taux de vrais positifs et le taux de faux positifs pour différents seuils de décision. Plus l'aire sous la courbe est proche de 1, plus le modèle distingue correctement les clients churners des clients non-churners.

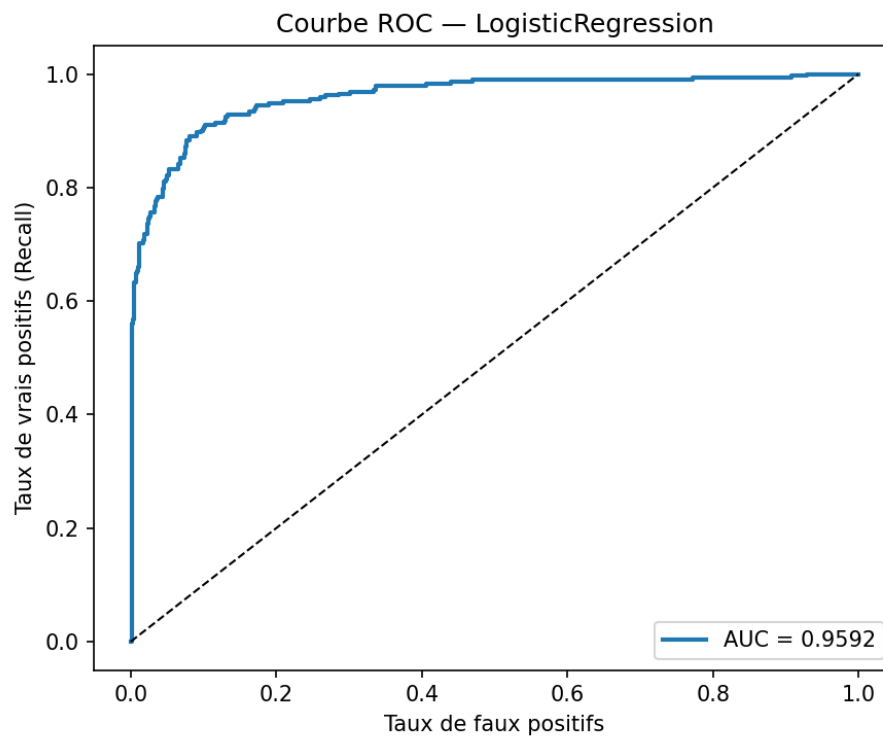


FIGURE 11 – Courbe ROC – Logistic Regression

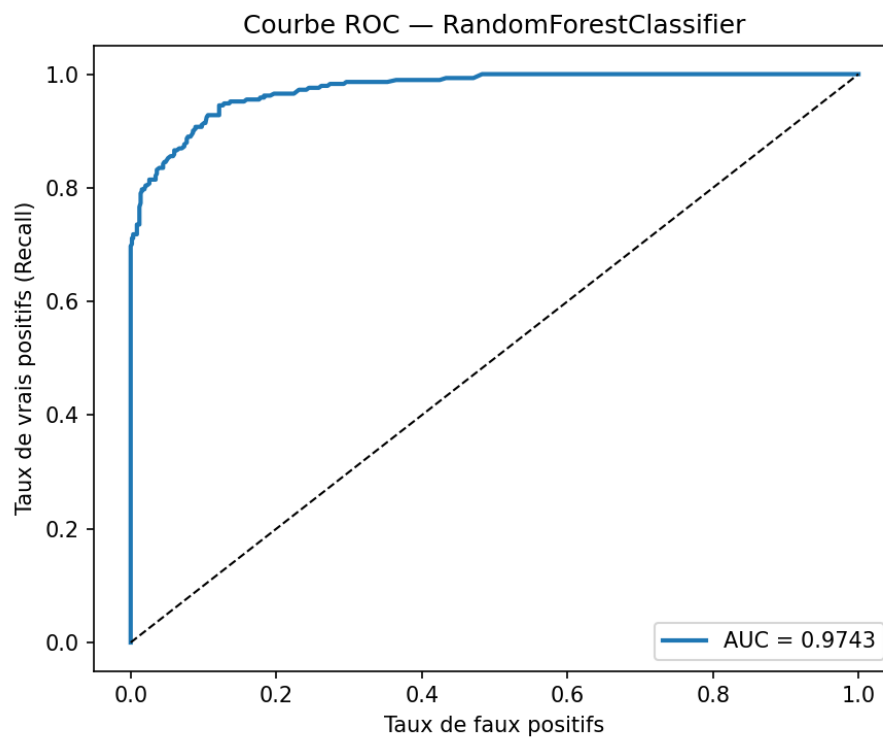


FIGURE 12 – Courbe ROC – Random Forest Classifier

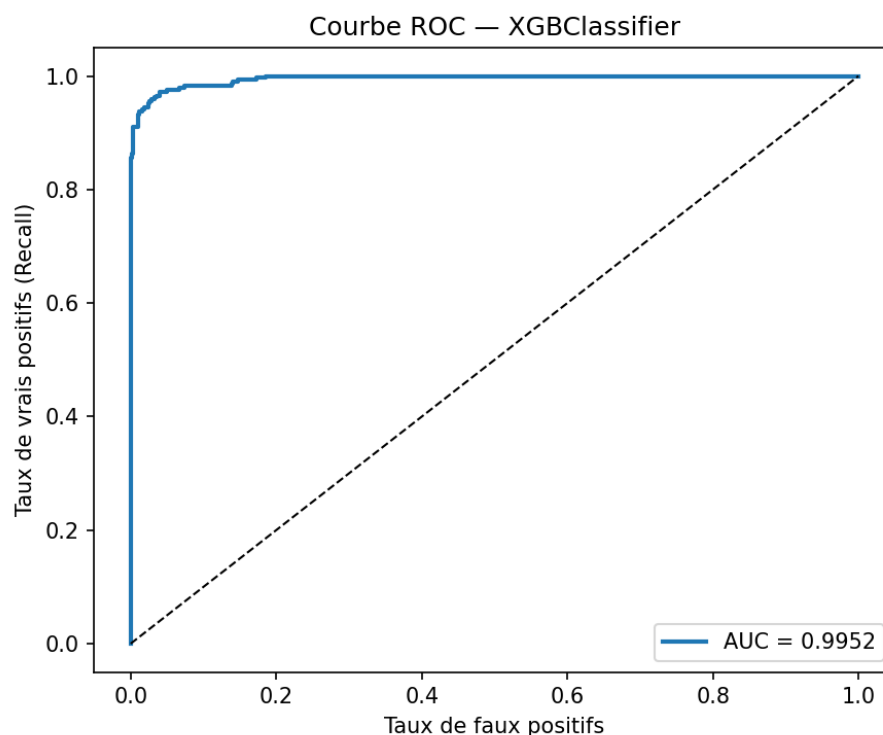


FIGURE 13 – Courbe ROC – XGBoost Classifier

La comparaison des courbes ROC confirme que les trois modèles distinguent correctement les deux classes. Cependant, XGBoost obtient la meilleure aire sous la courbe avec un **ROC-AUC de 0.9952**, suivi de Random Forest et de Logistic Regression. Cela indique que XGBoost possède la meilleure capacité de discrimination entre les clients churners et non-churners.

5.6 Importance des variables

Pour mieux interpréter le modèle retenu, l'importance des variables a été analysée. Cette étape permet d'identifier les facteurs les plus influents dans la prédiction du churn.

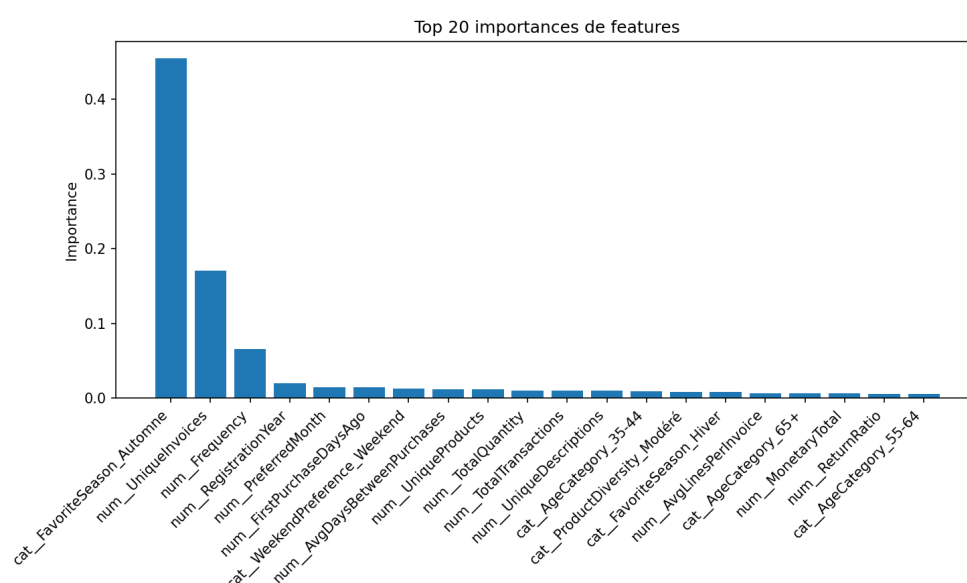


FIGURE 14 – Importance des variables – XGBoost Classifier

L'analyse de l'importance des variables permet de mieux comprendre les facteurs associés au risque

de churn. Elle contribue également à rendre les résultats plus exploitables d'un point de vue métier, notamment pour cibler les clients à risque et orienter les actions de fidélisation.

5.7 Modèle final retenu

Le modèle final retenu pour la classification du churn est **XGBoost Classifier**. Il obtient les meilleures performances globales avec une **Accuracy de 0.9703**, un **F1-score de 0.9545** et un **ROC-AUC de 0.9952**. Ces résultats montrent que le modèle est capable d'identifier efficacement les clients à risque tout en conservant une bonne précision globale.

Ce choix est justifié par sa capacité à modéliser des relations non linéaires entre les variables clients et la variable cible **Churn**. De plus, XGBoost utilise un apprentissage séquentiel par boosting, où chaque nouvel arbre corrige progressivement les erreurs des arbres précédents. Cela permet d'obtenir un meilleur équilibre entre la précision et le rappel, ce qui est particulièrement important dans un contexte de prédiction du churn.

5.8 Importance des variables

Pour mieux interpréter le modèle final retenu, l'importance des variables du modèle **XGBoost Classifier** a été analysée. Cette étape permet d'identifier les facteurs les plus influents dans la prédiction du churn et de rendre les résultats plus exploitables d'un point de vue métier.

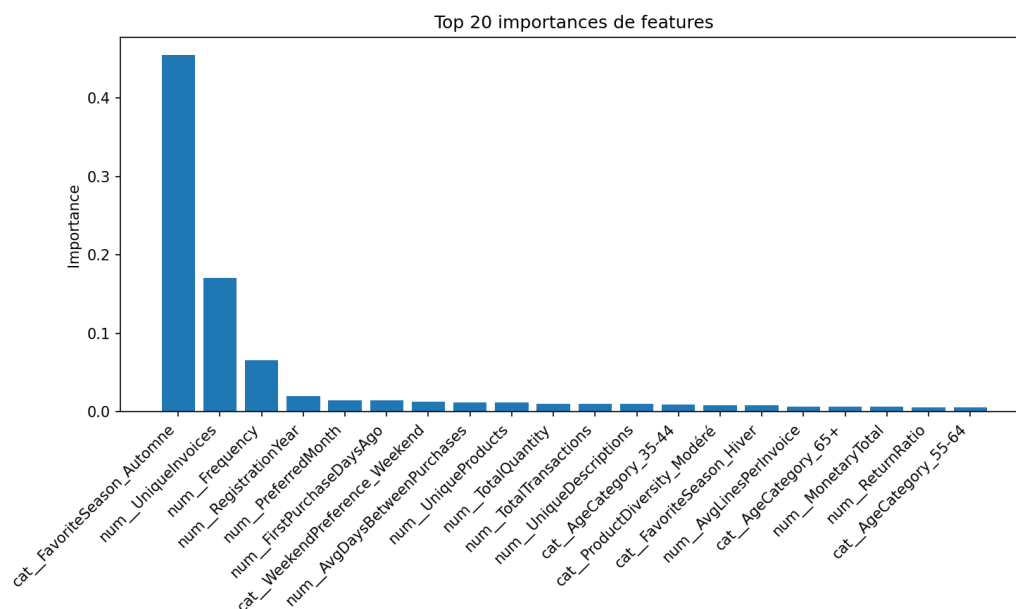


FIGURE 15 – Importance des variables – XGBoost Classifier

L'analyse de l'importance des variables montre quelles caractéristiques clients contribuent le plus à la décision du modèle. Les variables comportementales et transactionnelles, telles que la fréquence d'achat, le montant dépensé, le taux de retour, la satisfaction et les interactions avec le support client, peuvent influencer la probabilité de churn.

Il est important de noter que les variables présentant un risque de *data leakage*, comme **ChurnRiskCategory**, **RFMSegment**, **CustomerType**, **LoyaltyLevel**, **SpendingCategory** et **AccountStatus**, ont été supprimées avant l'entraînement du modèle. Cette suppression permet d'éviter que le modèle n'apprenne des raccourcis artificiels et garantit une interprétation plus réaliste des facteurs associés au churn.

Ainsi, l'importance des variables ne sert pas seulement à expliquer le fonctionnement du modèle, mais aussi à guider les actions métier. Elle peut aider l'entreprise à cibler les clients à risque, améliorer la satisfaction, réduire les retours et adapter les actions de fidélisation.

6 Modélisation – Clustering (Segmentation)

6.1 K-Means – Sélection du nombre optimal de clusters

Le script `src/clustering.py` applique l'algorithme *K-Means* sur les 36 variables numériques, après normalisation et sans utiliser la variable cible **Churn**. Le choix du nombre optimal de clusters repose sur trois critères complémentaires : la méthode du coude (*Elbow*), le score de silhouette et l'indice de Davies-Bouldin.

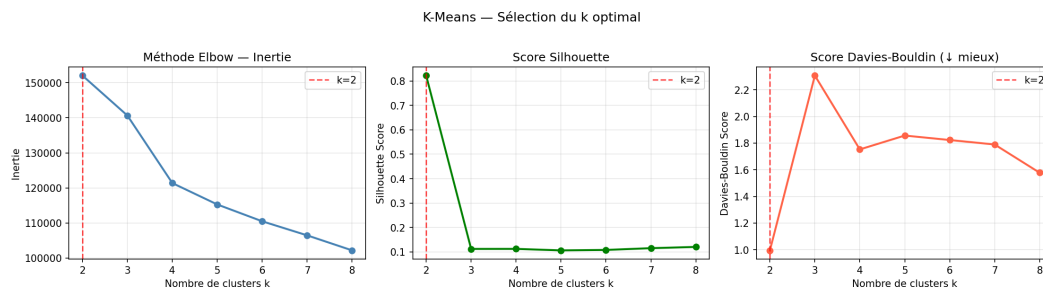


FIGURE 16 – Méthode Elbow, score de silhouette et indice de Davies-Bouldin pour la sélection de k

Le graphique *Elbow* montre un coude net pour $k = 2$. Le score de silhouette atteint sa valeur maximale pour $k = 2$ (0.826), ce qui indique une très bonne séparation entre les groupes. De plus, l'indice de Davies-Bouldin est minimal pour cette même valeur (0.982), ce qui confirme une structure de clustering satisfaisante. Ces trois critères concordent donc pour retenir $k = 2$ comme nombre optimal de clusters.

6.2 Profils des clusters K-Means

Les deux clusters identifiés par *K-Means* correspondent à des profils clients très distincts, comme le montre le tableau suivant.

TABLE 7 – Profil moyen des clusters obtenus par K-Means

Cluster	Clients	Recency	Frequency	MonetaryTotal	Churn	Profil
0 – VIP Champions	17 (0.4%)	4 jours	92 cmd	94 947 £	0.0%	Clients exceptionnels
1 – Standards	4 355 (99.6%)	92 jours	5 cmd	1 535 £	33.4%	Clients classiques

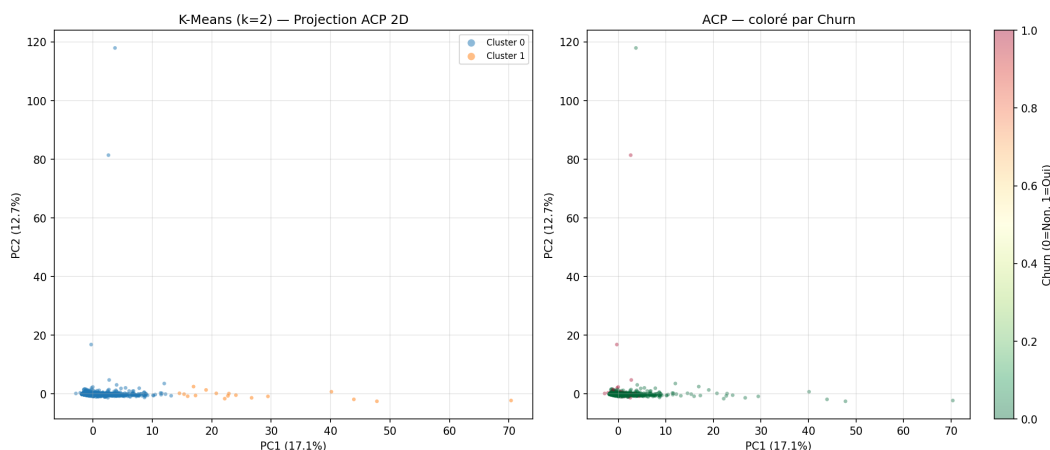


FIGURE 17 – Projection ACP 2D des clusters K-Means et superposition avec la variable Churn

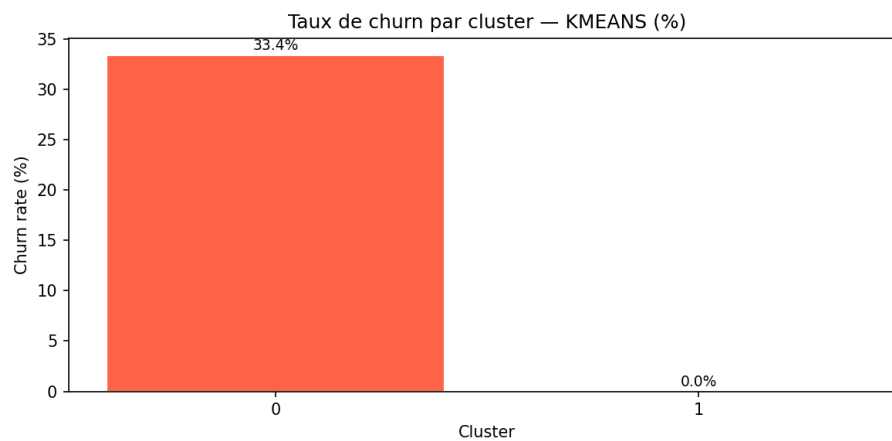


FIGURE 18 – Taux de churn par cluster K-Means

Le cluster 0 regroupe un très petit nombre de clients à forte valeur, caractérisés par une récence très faible, une fréquence d'achat très élevée et un niveau de dépense exceptionnel. Leur taux de churn est nul. À l'inverse, le cluster 1 correspond à la grande majorité des clients standards, avec une activité plus faible et un taux de churn nettement plus élevé. Cette segmentation met donc en évidence une opposition claire entre une clientèle VIP très fidèle et un ensemble de clients classiques plus exposés au risque de départ.

6.3 DBSCAN – Détection de clusters et d'outliers

En complément de *K-Means*, l'algorithme *DBSCAN* a été appliqué afin d'identifier des groupes de forme libre et de détecter les observations atypiques. Les paramètres retenus sont `eps=3.0` et `min_samples=5`, adaptés à des données préalablement normalisées.

TABLE 8 – Distribution des clusters détectés par DBSCAN

Cluster	Clients	MonetaryTotal	Churn	Profil / Interprétation
-1 (Outliers)	503 (11.5%)	7 436 £	27.4%	Clients atypiques à surveiller
0 (Principal)	3 806 (87.1%)	1 188 £	33.8%	Clients standards
1	11 (0.3%)	-161 £	100%	Inactifs depuis 1 an – perdus
2	15 (0.3%)	1 041 £	53.3%	Profil à risque élevé
3	7 (0.2%)	1 065 £	57.1%	Profil à risque élevé
4	17 (0.4%)	520 £	23.5%	Profil modéré
5	13 (0.3%)	529 £	7.7%	Clients fidèles actifs

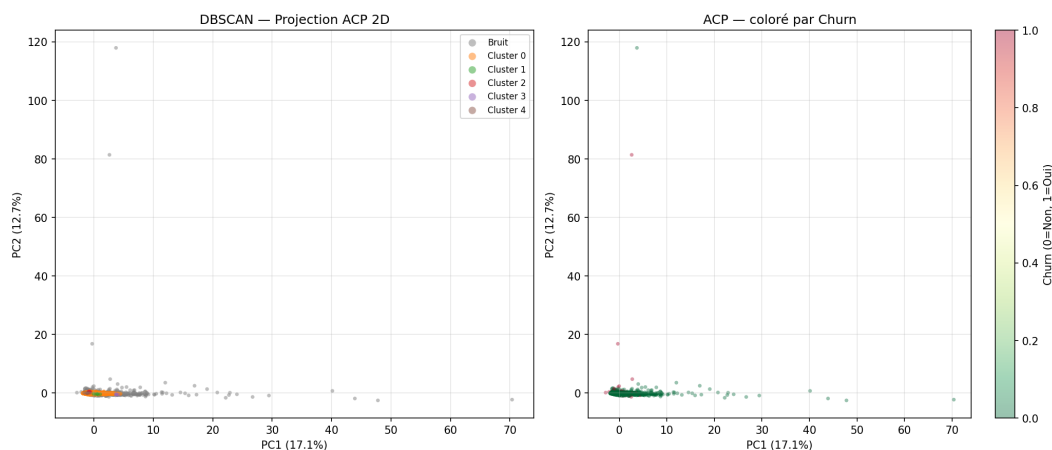


FIGURE 19 – Projection ACP 2D des clusters DBSCAN et du bruit

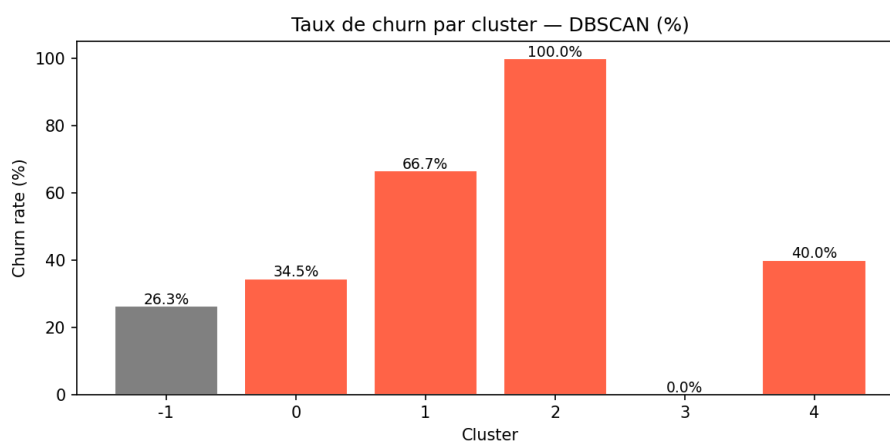


FIGURE 20 – Taux de churn par cluster DBSCAN

Les résultats montrent que *DBSCAN* détecte 6 clusters ainsi qu'un groupe important d'observations atypiques représentant 11.5% des clients. Le cluster 1 apparaît comme le plus critique, avec un taux de churn de 100% et une récence très élevée, ce qui correspond à des clients inactifs depuis longtemps. À l'inverse, le cluster 5 se distingue par un faible taux de churn, ce qui en fait un groupe de clients relativement fidèles. Cette approche permet donc de mettre en évidence des sous-profil plus fins que ceux obtenus avec *K-Means*, tout en isolant les comportements atypiques.

7 Modélisation – Régression (Dépense Future)

7.1 Objectif et préparation

Le script `src/regression.py` vise à prédire la variable `MonetaryTotal`, représentant la dépense totale future d'un client. Afin d'éviter toute fuite d'information (*data leakage*), les colonnes directement dérivées de cette variable cible sont exclues des variables d'entrée, notamment `MonetaryAvg`, `MonetaryStd`, `MonetaryMin`, `MonetaryMax`, `MonetaryPerDay` et `AvgBasketValue`.

À titre de rappel, deux modèles de régression sont comparés :

- **la régression linéaire**, utilisée comme modèle de base (*baseline*) ;
- **le Random Forest Regressor**, modèle plus flexible, capable de modéliser des relations non linéaires entre les variables.

Les caractéristiques de la variable cible sont les suivantes :

- variable cible : `MonetaryTotal` (dépense totale future en £) ;
- valeur minimale : -4287 £ ;
- valeur maximale : 279 489 £ ;
- moyenne : 1 898 £ ;
- médiane : 648 £.

Afin de conserver une cible cohérente pour l'apprentissage, seuls les clients ayant une valeur strictement positive de `MonetaryTotal` sont conservés, soit 4 320 clients. Le jeu de données est ensuite séparé en 80% pour l'apprentissage (3 456 clients) et 20% pour le test (864 clients).

7.2 Comparaison des modèles de régression

Les performances des deux modèles sont évaluées à l'aide des métriques MAE, RMSE et R^2 , ce dernier étant retenu comme critère principal de sélection.

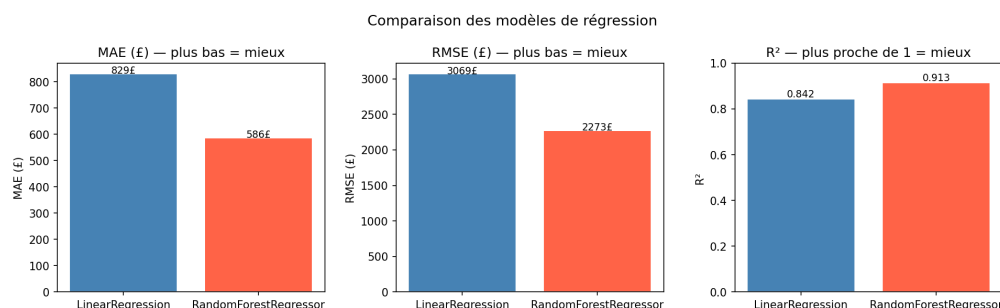


FIGURE 21 – Comparaison des métriques MAE, RMSE et R^2

TABLE 9 – Comparaison des modèles de régression sur le jeu de test

Modèle	MAE (£)	RMSE (£)	R^2	Critère	Sélection
Régression linéaire	819.10	3 099.40	0.8390	R^2	—
Random Forest Regressor	513.22	2 208.28	0.9183	R^2	Retenu

Les résultats montrent que le modèle *Random Forest Regressor* surpasse la régression linéaire sur l'ensemble des métriques. Il présente une erreur absolue moyenne plus faible, une meilleure robustesse face aux grandes erreurs, ainsi qu'un coefficient de détermination plus élevé. Il a donc été retenu comme modèle final pour la prédiction de la dépense future.

7.3 Valeurs réelles vs valeurs prédites

L'analyse graphique des prédictions permet d'évaluer la qualité d'ajustement des modèles. Un bon modèle de régression produit généralement des points proches de la diagonale idéale et des résidus centrés autour de zéro.

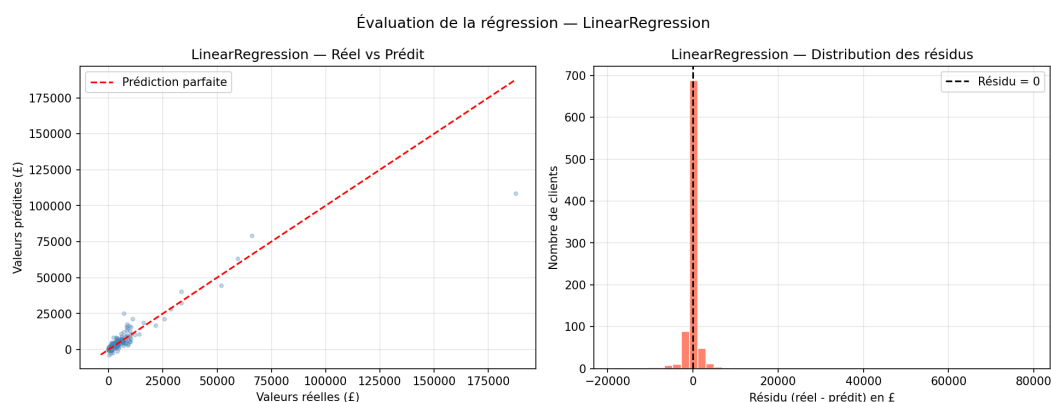


FIGURE 22 – Régression linéaire – Valeurs réelles vs prédites et résidus

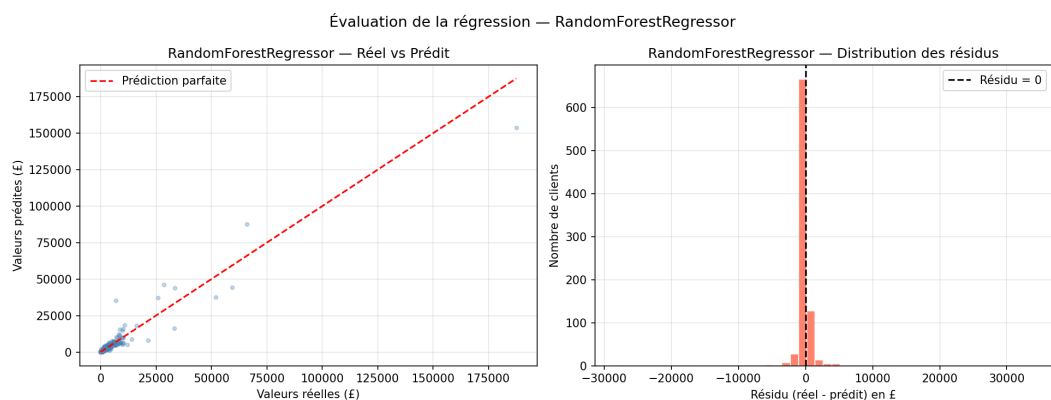
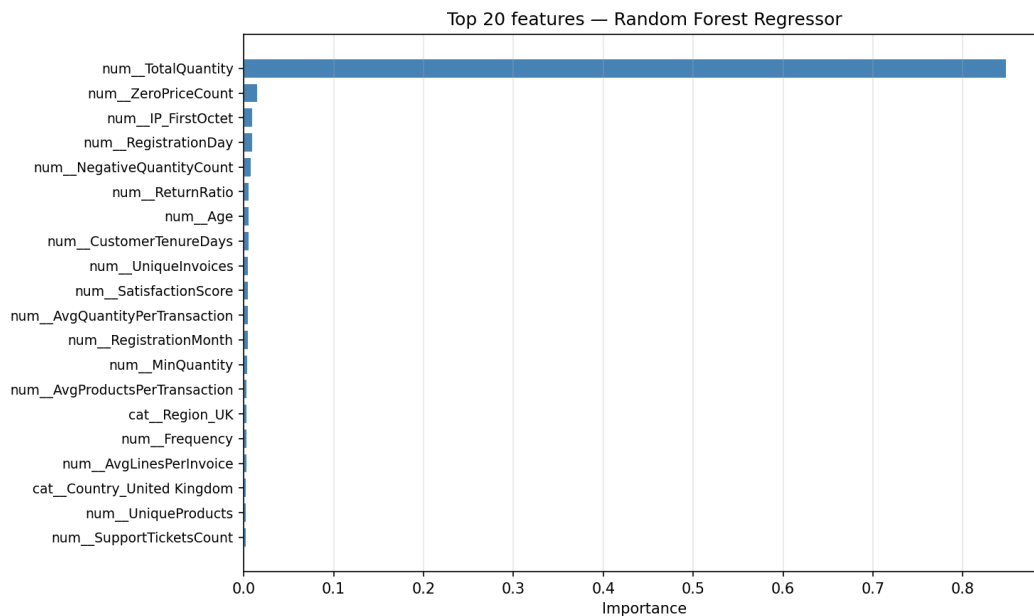


FIGURE 23 – Random Forest Regressor – Valeurs réelles vs prédites et résidus

Le modèle *Random Forest Regressor* montre une meilleure adhérence à la diagonale et des résidus plus concentrés autour de zéro. Cela indique que la majorité des prédictions produites sont proches des valeurs réelles, avec une dispersion globalement plus faible que celle observée pour la régression linéaire.

7.4 Importance des variables pour la régression

Le modèle *Random Forest Regressor* permet également d'estimer l'importance relative des variables dans la prédiction de la dépense future.

FIGURE 24 – Top 20 des variables les plus importantes pour la prédiction de **MonetaryTotal**

L'analyse montre que la variable **TotalQuantity** domine largement les autres variables, avec une importance très élevée. Cela signifie que la quantité totale commandée constitue le principal facteur explicatif de la dépense future. Les autres variables, telles que certaines catégories de dépense ou des variables démographiques, jouent un rôle secondaire. Cette hiérarchie suggère une relation forte entre volume d'achat et montant total dépensé.

7.5 Exemples de prédictions

Le tableau suivant présente quelques exemples de prédictions réalisées sur le jeu de test par le modèle retenu.

TABLE 10 – Exemples de prédictions sur quelques clients du jeu de test

Client	Réel (£)	Prédit (£)	Écart (£)	Précision
1	3 385.62	3 166.48	219.14	93.5%
2	8 574.11	11 534.46	2 960.35	65.5%
3	3 189.81	2 909.23	280.58	91.2%
4	1 649.36	1 232.25	417.11	74.7%
5	962.39	881.42	80.97	91.6%

Ces exemples confirment que le modèle retenu fournit, dans la majorité des cas, des estimations proches des valeurs réelles, même si certaines erreurs plus importantes peuvent apparaître pour des profils clients particuliers.

8 Déploiement – Interface Flask

8.1 Architecture de l'application

L'application Flask, implémentée dans le fichier `app/app.py`, constitue la couche de déploiement du projet. Elle expose une API REST permettant d'interroger les différents modèles entraînés et sert également une interface web interactive destinée à faciliter leur utilisation.

L'objectif de cette application est de rendre opérationnels les modèles développés dans les chapitres précédents, en offrant un accès simple aux fonctionnalités de classification, de régression et de clustering à travers un tableau de bord unique.

TABLE 11 – Principales routes de l'API Flask

Méthode	Route	Description
GET	/	Dashboard web interactif organisé en plusieurs onglets
GET	/health	Vérification de l'état des modèles et des artefacts
GET	/api	Informations générales sur l'API au format JSON
POST	/predict	Prédiction du churn (0/1) avec probabilité associée
POST	/predict_revenue	Prédiction de la dépense future du client
POST	/predict_cluster	Identification du segment client par K-Means
POST	/predict_all	Exécution simultanée des trois prédictions

8.2 Dashboard web

Le dashboard, défini dans le fichier `app/templates/index.html`, propose une interface web organisée par type de modélisation. Cette interface permet à l'utilisateur de saisir les informations d'un client, de lancer les prédictions et d'obtenir une restitution visuelle des résultats.

Le tableau de bord est structuré en quatre onglets :

- **Onglet 1 – Classification (Churn)** : formulaire de saisie des variables du client, affichage du résultat final (*fidèle* ou *churn*) ainsi qu'une probabilité associée et un historique des dernières prédictions ;
- **Onglet 2 – Régression (Dépense future)** : formulaire de saisie permettant d'estimer le montant futur dépensé par le client, avec restitution du niveau de dépense prédit ;
- **Onglet 3 – Clustering (Segment)** : identification du segment client à partir des variables numériques et affichage du profil correspondant ;
- **Onglet 4 – Tout en un** : exécution simultanée des trois types de prédictions à partir d'un seul formulaire.

Cette interface améliore l'accessibilité des modèles et permet d'illustrer concrètement leur utilisation dans un cadre applicatif.

8.3 Gestion des colonnes

Une difficulté importante du déploiement réside dans l'écart entre les variables directement saisies par l'utilisateur dans le formulaire et l'ensemble des colonnes attendues par le préprocesseur entraîné. Pour résoudre ce problème, une fonction `align_columns()` est utilisée afin d'assurer la compatibilité entre les entrées utilisateur et le pipeline de prétraitement.

Cette fonction ajoute automatiquement les colonnes manquantes avec des valeurs `NaN`, lesquelles sont ensuite prises en charge par les mécanismes d'imputation du pipeline *sklearn*. Cette approche garantit une cohérence entre l'environnement d'entraînement et l'environnement de prédiction.

8.4 Lancement de l'application

L'application peut être démarrée depuis la racine du projet à l'aide de la commande suivante :

```
1 python app/app.py
```

Listing 2 – Lancement de l'application Flask

Une fois l'application lancée, l'interface web est accessible dans le navigateur à l'adresse suivante :

```
1 http://127.0.0.1:5000
```

Ainsi, le déploiement via Flask permet de transformer les modèles développés en une solution interactive, exploitable par un utilisateur final sans manipulation directe du code Python.

9 Conclusion

Ce projet a permis de mettre en œuvre une chaîne complète de Machine Learning appliquée à l'analyse comportementale de la clientèle retail, depuis l'exploration des données brutes jusqu'au déploiement d'une application web interactive. Il a couvert les trois principales tâches de modélisation étudiées dans ce travail : la classification du churn, la segmentation de la clientèle par clustering et la prédiction de la dépense future par régression.

Les résultats obtenus montrent des performances particulièrement élevées. En classification, le modèle *Random Forest* a obtenu les meilleurs résultats, avec une accuracy et une ROC-AUC égales à 1 sur le jeu de test. En clustering, *K-Means* a permis d'identifier deux grands profils de clients, tandis que *DBSCAN* a mis en évidence plusieurs sous-groupes ainsi qu'un ensemble d'observations atypiques. En régression, le modèle *Random Forest Regressor* a fourni les meilleures performances, avec un coefficient de détermination $R^2 = 0.9183$ et une erreur absolue moyenne de 513.22 £.

Ces résultats doivent néanmoins être interprétés avec prudence. En effet, les scores parfaits obtenus en classification s'expliquent en partie par la nature synthétique du dataset utilisé, dans lequel certaines variables telles que `RFMSegment` ou `LoyaltyLevel` sont fortement liées à la variable cible `Churn`. Dans un contexte réel de production, les performances attendues seraient généralement plus modérées.

Au-delà des résultats chiffrés, ce projet a permis de consolider plusieurs compétences importantes, notamment l'analyse exploratoire des données, la préparation rigoureuse des variables, la prévention du *data leakage*, l'évaluation comparative de plusieurs modèles et leur intégration dans une application déployable. Il illustre ainsi l'ensemble du cycle de vie d'un projet de Machine Learning, de la donnée brute jusqu'à l'usage applicatif.

Pistes d'amélioration

Plusieurs améliorations peuvent être envisagées afin d'enrichir ce travail :

- optimisation des hyperparamètres à l'aide de méthodes comme `GridSearchCV` ou `Optuna` ;
- utilisation de modèles plus avancés pour les données tabulaires, tels que `XGBoost` ou `LightGBM` ;
- recours à des techniques de rééquilibrage de classes comme `SMOTE` ;
- amélioration de l'interprétabilité des prédictions à l'aide de méthodes comme `SHAP` ;
- mise en place d'un suivi en production permettant de détecter d'éventuels changements dans la distribution des données.

10 Références

1. Scikit-learn Documentation. <https://scikit-learn.org/stable/>
2. Pedregosa, F. et al. (2011). *Scikit-learn : Machine Learning in Python*. Journal of Machine Learning Research, 12, 2825–2830.
3. Breiman, L. (2001). *Random Forests*. Machine Learning, 45(1), 5–32.
4. Ester, M. et al. (1996). *A density-based algorithm for discovering clusters in large spatial databases with noise*. KDD.
5. Rousseeuw, P. J. (1987). *Silhouettes : a graphical aid to the interpretation and validation of cluster analysis*. Journal of Computational and Applied Mathematics, 20, 53–65.
6. Davies, D. L., & Bouldin, D. W. (1979). *A Cluster Separation Measure*. IEEE Transactions on Pattern Analysis and Machine Intelligence, 1(2), 224–227.
7. Flask Documentation. <https://flask.palletsprojects.com/>
8. Pandas Documentation. <https://pandas.pydata.org/docs/>